

SOVEREIGN · GOVERNED · SELF-HOSTABLE

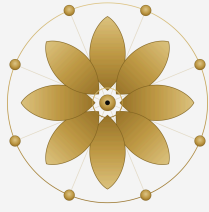


Building Agentic Systems

Design, build, run, and evaluate a governed AI team — in five phases.

Data Masterclass • datamasterclass.com

A practical guide • July 2026



SOVEREIGN AGENTIC OS

Contents

What you are about to build

The mental model (read this first)

Before you start

Phase 1 · Define — write the brief

Phase 2 · Design — shape the team

Phase 3 · Build — compile and verify

Phase 4 · Run — execute the team

Phase 5 · Evaluate — prove it worked

How the guardrails actually work

Automate it

Building from Claude, over MCP

A worked example

Best practices

In closing

What you are about to build

An **agentic system** in the Sovereign Agentic OS is a small **team of AI agents** that work together on a real task – reading your data, consulting your knowledge, and (when you allow it) proposing or making changes – and then hand you a result you can trust. It is not a chatbot and it is not a single prompt. It is a governed team with named roles, a defined job, and a boundary you draw around what it may touch.

Three things make it different from wiring an agent up by hand:

- **It runs as you, under policy.** Every action the team takes is executed as the signed-in user, checked by the policy engine (OPA), filtered by row- and document-level security, and written to the audit trail. The agent path is the *same governed path* as the user interface – never a back door.
- **A grant is a capability.** You do not hand agents raw tools and hope. You grant the team specific context – a dataset, a knowledge folder, a metric – and that grant *is* the capability. Agents can use what you granted and nothing else.
- **You build it in five phases.** The builder walks you from a blank idea to an evaluated, repeatable team: **Define → Design → Build → Run → Evaluate**. You can stop, inspect, and iterate at every phase.

There are two ways to work. **Simple mode** is the guided, no-code path – describe the job, adjust a few cards, press run. **Developer mode** exposes the full agent graph and raw configuration for people who want it. This guide follows Simple mode, which is what most teams should use; everything here maps one-to-one onto Developer mode.

The mental model (read this first)

Five ideas carry the whole system. Hold them and the rest is mechanics.

1. **A team, not a bot.** An agentic system is one or more agents, each with a role (an *Analyst*, a *Recommender*, a *Reviewer*). They pass work between them and finish with a single result.
2. **Context you grant is the only context they get.** Agents are grounded exclusively in what you grant in the Define phase – your knowledge, files, data, connections, metrics, and plan items. Nothing else leaks in.
3. **A grant is a default-on capability.** The moment you grant a resource, the team gains the ability to use it and the capability shows up on the agent. There is no second step where you "enable the tool" – the grant is the enablement.
4. **One safety posture for the whole team.** The team carries a single *safety preset* that caps what any grant can do – from read-only, through propose-and-approve, up to bounded direct writes. You cannot accidentally exceed it on one item.
5. **Governed end to end.** It runs as you, sees only your rows, and records everything. Sovereignty and safety are not switches you remember to flip; they are the default.

Before you start

Build the context first, then the team. An agentic system is only as good as what it can see, so make sure the material it needs already lives in the OS:

- **Data** – the datasets it will analyze, ideally promoted to at least Silver or Gold in the Data tab.
- **Knowledge & Files** – the reference material and documents it should reason from, in the Knowledge and Files tabs.
- **Connections & Metrics** – any external systems or business metrics it must read.
- **Plan items** – the Strategy pillars, Big Bets, Operating Model, or Workflows that give it direction.

If something is missing, create it in the relevant Context or Plan tab first – you will be able to grant it in a moment. Then open the **Agents** tab and choose **New** to start the builder.

Phase 1 · Define — write the brief

Define is where you tell the OS *what the team is for* and *what it is allowed to touch*. Take your time here; a sharp brief makes every later phase easier.

Name and describe. Give the system a clear name, then use the description box to *describe the job in plain language* — what should the team accomplish, with what inputs, and what should the output look like. The OS reads this and scaffolds a starting team for you, which you refine in Design.

Choose the safety preset. This single setting is the team's safety posture, and it caps every grant:

- **Read-only** — the team can read its granted context but change nothing. The safest starting point.
- **Read + propose** — the team can *propose* changes, which are held in the Policies & Approvals queue for a human to approve. The sensible default for real work.
- **Read + bounded write** — the team can write directly, but only inside its own workspace, with no approval. Introducing this posture is reserved for builders.
- **Full in scope** — every granted item may be written directly. Use deliberately.

Start low. You can always raise the posture once you trust the team.

Pick a trigger. Decide how the team runs: **manually** (you press run), on a **schedule** (a cron cadence — nightly, weekly), or on an **event**. Manual is the right choice while you are still designing.

Grant "What your team can use." This is the heart of Define. You choose exactly which resources the team may use, grouped into two families:

- **Plan Items** — Strategy, Big Bets, the Operating Model, and Workflows. These give the team direction and business context.
- **Context** — Knowledge, Files, Data, Connections, and Metrics. This is the material the team reasons over and acts on.

For each item you grant, you set an **access level** — *read-only*, *read + propose*, or *read + write* — and the OS shows you the scope (My / Domain / Company) of what you are granting. The safety preset caps these levels, and the interface tells you honestly when a level is locked and why. Remember: **each grant you add is automatically a capability** the team can use — you will see it reflected on the agents in the next phase.

State the evaluation up front. A good task tells the team what "done" means. Phrase it so it names the **deliverable** (what to produce), the **data** (what to use), the **rule** (any constraint), and the **format** (how to present it). You will reuse this in the Evaluate phase to score the run.

When the brief is right, add the team and move to Design.

Phase 2 · Design — shape the team

Design is where the scaffolded team becomes *your* team. Each agent is a card you can edit.

Each agent has a role, instructions, and capabilities. The **role is the agent's name** — call it what it does (*Campaign Analyst*, *Budget Recommender*) and that label follows the agent through Run, Evaluate, the team graph, and the reports. Give it clear **instructions**: what this agent is responsible for and how it should behave. Its **capabilities** appear as chips, scoped strictly to the team's grants from Define — by default the OS sets these to **Auto**, giving each agent what it needs from what you allowed, and you can narrow them per agent.

Add or remove agents. Use **+ Add agent** to bring in another role; a template picker offers sensible archetypes to start from. Keep the team as small as the job allows — two or three focused agents usually beat one agent doing everything or a crowd that talks past each other.

How they connect. In Simple mode the agents are wired in a clean linear hand-off automatically — each finishes and passes to the next. Developer mode exposes the full graph if you need branches, a supervisor, or parallel work. A single-agent system is perfectly valid for a focused task; reach for multiple agents when the work has genuinely separate steps (gather, then judge, then recommend).

Models are chosen for you. The OS picks an appropriate model per agent — a fast model for gathering, a reasoning model for judgement and synthesis — and you can override it. You do not have to think about this to get a good result.

Phase 3 · Build — compile and verify

Build turns your design into a runnable, governed team. When you start it, a progress indicator walks the real steps with live commentary: it **provisions the exact tools and grants** each agent is allowed to call, **wires the graph**, links tracing, and **commits the team's files**.

A green build means every agent resolved to real, permitted capabilities and the team is ready to run. If a step fails, the report tells you which one and why — usually a grant that needs adjusting back in Define — so you can fix it and rebuild. Nothing runs against your data during Build; it only assembles and checks.

Phase 4 · Run — execute the team

Press ► **Run** and the team goes to work. You are not left staring at a spinner:

- **Live progress** streams the current step so you can see which agent is working and on what.
- **Per-agent trace and output** let you open any node and read exactly what it did — the tools it called, what it read, what it produced.
- **The final result** is presented in full, with real tables rendered from the data.

When you are happy with a run, use **Download PDF Results Report** to export exactly what is on screen — the run summary, the final output, and each agent's result — as a clean, shareable document.

Phase 5 · Evaluate — prove it worked

A result you cannot trust is not a result. Evaluate is where the OS holds the run up to the light.

- **Deterministic checks** verify the objective, factual things — did the team produce the deliverable, in the required format, respecting the rule you set in the brief?
- **An AI judge** scores the qualitative dimensions and explains its reasoning.
- **Context actually used** shows, per agent, the real files, data, knowledge, metrics, and connections each one consumed during the run — with deep links to the exact artifacts, so you can check the sources yourself. It also surfaces **dead grants**: context you granted that the team never touched, so you can tighten the next version.

Export the whole assessment with **Download PDF Evaluation Report** — it opens with the visual multi-agent graph, then the on-screen evaluation, followed by three appendices: the results, a summary of your Define-stage settings and inputs, and each agent's description. Read the evaluation, adjust the brief or the team, and run again. Iteration here is how a rough first team becomes a dependable one.

How the guardrails actually work

It is worth understanding *why* you can hand a team access to real systems and sleep at night.

- **Grants are the boundary.** A team can only use what you granted – every tool call is checked against the grant set and the policy engine before it runs.
- **The preset caps everything.** No single item can exceed the team's safety posture, so a permissive grant on one dataset cannot quietly become a write if the team is read-only.
- **Propose means a human decides.** At *read + propose*, the team's writes are not applied – they are queued in **Policies & Approvals** for a person to approve or reject.
- **Direct writes are bounded and builder-gated.** *Read + write* applies changes immediately, but only within the team's own workspace, and only a builder may introduce that posture – a creator's attempt to craft a direct-write grant is rejected in favour of the propose path.
- **Everything is you, and everything is logged.** The team acts as the signed-in user, sees only the rows and documents that user may see, and writes every action to the audit trail.

The result: agents get real reach without becoming a real risk.

Automate it

Once a team is proven, it does not have to be run by hand. In Define you can set it to run on a **schedule** (a cron cadence) or in response to an **event**, so a weekly analysis or a triggered response happens on its own. Results are captured just as they are for a manual run, and notifications land in your inbox – with the same governance, tracing, and evaluation applied to every automated run.

Building from Claude, over MCP

Everything in this guide is also available from Claude (or any MCP client) through the OS's Model Context Protocol connection – the *same governed path*, running as you. You can discover what you are allowed to do with `list_capabilities`, assemble a team with `build_agent_system`, inspect it with `get_agent_system`, and execute it with `run_agent_system`. The grants, the safety preset, and the audit trail behave identically whether you build in the tab or over MCP.

A worked example

Suppose you want a team that recommends how to reallocate next week's marketing budget across campaigns.

- **Define.** Name it *Campaign Budget Optimizer*. Describe the job: "Analyze last week's campaign performance and recommend budget shifts to improve margin ROI, respecting each campaign's guardrails; present a ranked table with a one-line rationale per campaign." Set the safety preset to **read + propose**. Grant the **campaign performance dataset** (read-only) and the **marketing playbook knowledge folder** (read-only); if the team should file its recommendation, grant the destination at **read + propose**. Trigger: manual.
- **Design.** Keep two agents. A **Campaign Analyst** reads the dataset and scores each campaign on spend, margin, and return rate. A **Budget Recommender** takes that scoring and the playbook and proposes reallocations, ranked, with rationale.
- **Build.** Compile and verify – the Analyst resolves to the dataset read tools, the Recommender to the knowledge read tools, and the team is ready.
- **Run.** Execute. Watch the Analyst score the campaigns, then the Recommender produce the ranked table. Export the results PDF.
- **Evaluate.** Confirm the deliverable is a ranked table with rationales (deterministic check), read the judge's take on the quality of the reasoning, and check that the Recommender actually used the playbook (context-used). Tighten and re-run if needed.

That is a complete, governed, repeatable agentic system – built in an afternoon, safe to point at real money.

Best practices

- **Start read-only or propose.** Prove the team's judgement before you let it write. Raise the posture later.
- **Grant the minimum.** Every grant is reach; give the team exactly what the job needs and nothing more. Let Evaluate's dead-grant view help you trim.
- **Write a sharp brief.** Name the deliverable, the data, the rule, and the format. A vague task produces a vague team.
- **Keep roles clear and few.** Two or three well-instructed agents beat a crowd. Split work only where the steps are genuinely distinct.
- **Iterate through Evaluate.** The first run is a draft. The evaluation tells you what to fix; that loop is where quality comes from.
- **Promote when proven.** A team that works is an asset — share it to your Domain or certify it Company-wide so others can build on it.

In closing

The Sovereign Agentic OS turns "let an AI do it" from a leap of faith into an engineering practice. You define the job and the boundary, design a small team, build and verify it, run it with full visibility, and evaluate it against a standard you set — all as yourself, under policy, on infrastructure you control. Start small, keep the grants tight, and let the Evaluate loop do the teaching. Your first dependable agentic team is a few phases away.